Sl.NO 1: Develop a program to create histograms for all numerical features and analyze the distribution of each feature. Generate box plots for all numerical features and identify any outliers. Use California Housing dataset.

Pandas is a fast, powerful, and open-source Python library designed for data manipulation, cleaning, and analysis.

Seaborn is a high-level Python library specifically designed for creating statistical data visualizations.

1. Understanding the Dataset: housing.csv

- **What is it? A table containing information about houses in California (like age of the house, number of rooms, and price).**
- **What is it for? We use it to see "patterns." For example, do expensive houses have more rooms? Is the data "clean" or does it have errors (outliers)?**
- **What is an Outlier? Imagine 9 people in a room earn $\$50,000$ and one person earns $\$5,000,000$. That $\$5$ million is an outlier—it's a data point that is very different from the others.**

2. The Code: Step-by-Step with "Plain English" Explanations

Cell 1: The Setup (Importing Tools)

In Python, we don't write everything from scratch. We "import" toolboxes.

Python

```
import pandas as pd       # Toolbox for handling tables (like Excel). We call it 'pd' for short.
import numpy as np        # Toolbox for math and numbers. We call it 'np'.
import matplotlib.pyplot as plt # Toolbox for basic drawing/plotting.
import seaborn as sns     # Toolbox for making plots look beautiful.
import warnings           # Toolbox to handle "warning" messages.


warnings.filterwarnings('ignore') # This tells the computer: "Don't show me minor warnings, just run the code."
# Load the data. 'df' stands for 'DataFrame' (think of it as a digital spreadsheet).
# Note: In the lab, just use 'housing.csv' if the file is in the same folder.
df = pd.read_csv("housing.csv")
```

```
df.head()    # Shows the first 5 rows to see what the data looks like.

df.shape     # Shows (Rows, Columns). Tells you how big the dataset is.

df.info()    # Shows column names and tells you if any data is missing.

df.nunique() # Counts how many unique values are in each column.
```

Cell 2: Cleaning the Data

Data is often "dirty" (missing values or duplicates). We must fix this.

Python

```
df.isnull().sum()    # Checks every column to see how many empty spaces (NaN) are there.

df.duplicated().sum() # Checks if any rows are repeated exactly.


# 'total_bedrooms' has some missing values. We fill them with the 'Median'.

# The Median is the middle value. It is better than the Average because it ignores outliers.

median_value = df['total_bedrooms'].median()

df['total_bedrooms'].fillna(median_value, inplace=True) # 'inplace=True' means save the change permanently.
```

Cell 3: Changing Data Types

Sometimes numbers are stored as "40.0" (float) when they should be "40" (integer).

Python

```
# This loop looks at columns 2 to 6 and turns them into whole numbers (integers).

for col in df.columns[2:7]:

    df[col] = df[col].astype(int)
```

Cell 4: Summary Statistics

Python

```
df.describe().T # Shows Mean, Median, Min, Max for every column. .T (Transpose) flips the table to read it better.
```

Identify only the columns that contain numbers (Numerical features)

Numerical = df.select_dtypes(include=[np.number]).columns

print(Numerical)

Cell 5: Creating Histograms (Frequency Distribution)

A Histogram tells you how many times a certain value appears.

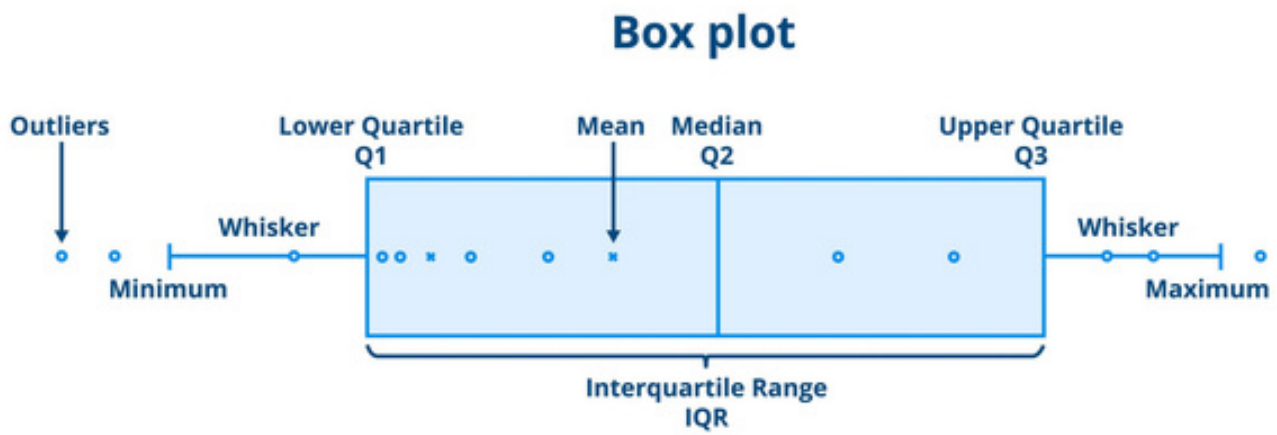
A **histogram** in Python is a graphical representation used to visualize the distribution of numerical data. It groups data points into continuous, non-overlapping intervals called **bins** and displays the frequency (count) of values within each bin using vertical bars.

Python

```
for col in Numerical:
    plt.figure(figsize=(10,6)) # Set the size of the drawing paper.
    df[col].plot(kind='hist', bins=60, edgecolor='black') # Draw bars. 'bins=60' means divide the range into 60 bars.
    plt.title(col)          # Add the name of the feature as the title.
    plt.ylabel('Frequency')  # Label the Y-axis (how many times it occurs).
    plt.show()              # Display the drawing.
```

Cell 6: Creating Box Plots (Identifying Outliers)

A Box Plot is a box with "whiskers." The points far away from the whiskers are Outliers.



Python

```
for col in Numerical:
    plt.figure(figsize=(6,6))
    sns.boxplot(x=df[col], color='blue') # Create the box plot.
    plt.title(col)
    plt.show()
```

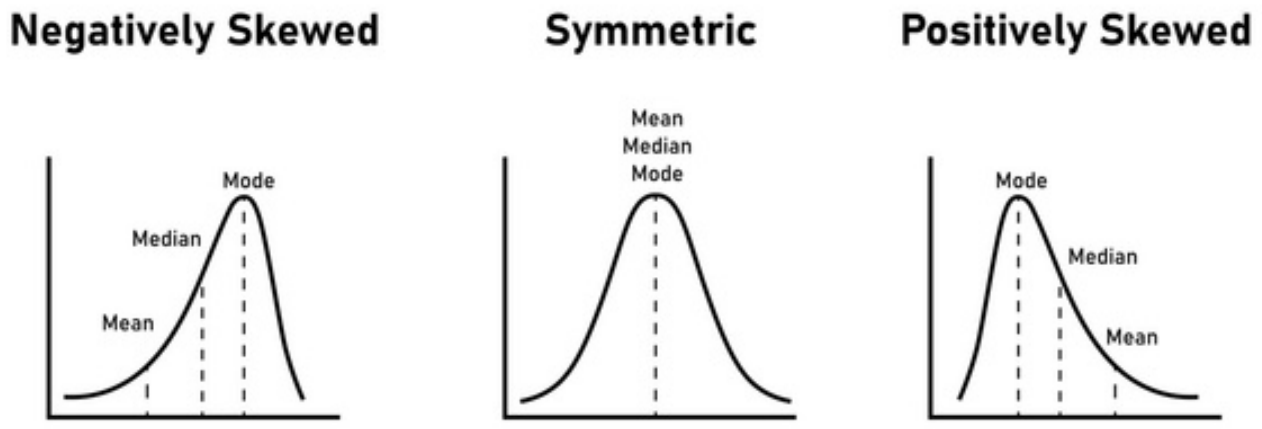
3. Key Terms Dictionary

Keyword	Simple Definition	GATE/Technical Logic
import	Load a toolbox.	Includes pre-written libraries into your script.
df	Your spreadsheet.	Short for "DataFrame," the primary object in Pandas.
pd.read_csv	Open the file.	Function to parse a Comma Separated Values file.
isnull()	Find empty spots.	Detects missing data points (NaN).
median()	The middle number.	A measure of central tendency robust to outliers.
fillna()	Patch the holes.	"Imputation" - filling missing data with estimated values.
hist	Histogram.	Visualizes the Probability Distribution of data.
boxplot	Box plot.	Visualizes the Five-Number Summary (Min, Q1, Median, Q3, Max).

4. Understanding the Shapes (The Logic)

- **Normal Distribution: The histogram looks like a bell. Most values are in the middle.**

- Skewed Distribution: The histogram has a long "tail" on one side.



- Box Plot Logic: * The middle line is the Median.
  - The box contains the middle 50% of the data.
  - The dots outside the lines are the Outliers. In the lab, if you see dots far to the right in median_income, those are the very rich districts.

Slide 1: The Mission

Sl.NO 2: Develop a program to Compute the correlation matrix to understand the relationships between pairs of features. Visualize the correlation matrix using a heatmap to know which variables have strong positive/negative correlations. Create a pair plot to visualize pairwise relationships between features. Use California Housing dataset.

- Goal: Find out how features are "connected." If one goes up, does the other go up too?
- The Dataset: California Housing data (built-in via Scikit-Learn).

Slide 2: The Logic (What is Correlation?)

Before the code, you must understand the "Correlation Coefficient (r)":

- Value is +1: Perfect relationship. As Income increases, House Price increases. (Strong Positive)
- Value is -1: Inverse relationship. As house age increases, value might decrease. (Strong Negative)

- **Value is 0: No relationship. (e.g., longitude has nothing to do with number of bedrooms).**

Slide 3: Code Breakdown (Cell 1 - Imports & Loading)

Code:

Python

```
import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing


# Step 1: Load the California Housing Dataset

california_data = fetch_california_housing(as_frame=True)

data = california_data.frame

data.head()
```

Explanation for Beginners:

- **import seaborn as sns: Seaborn is a "Beauty Toolbox." It takes the boring graphs from Matplotlib and makes them professional with colors.**
- **fetch_california_housing: Instead of reading a .csv file from your computer, this command "calls" the data directly from the Python library's internal storage.**
- **as_frame=True: This tells the computer, "Give me this data in a table format (DataFrame) so I can use it with Pandas."**

Slide 4: Code Breakdown (Cell 2 - The Heatmap)

Code:

Python

```
# Step 2: Compute correlation matrix

correlation_matrix = data.corr()


# Step 3: Heatmap Visualization

plt.figure(figsize=(10,8))
```

```
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)

plt.title('Correlation Matrix of California Housing Features')

plt.show()
```

Explanation for Beginners:

- **data.corr(): This is the "Brain" of the program. It calculates the math formula for correlation for every possible pair of columns.**
- **sns.heatmap(...):**
  - **annot=True: This puts the actual numbers inside the colored boxes.**
  - **cmap='coolwarm': This sets the color theme. Warm (Red) = High Correlation; Cool (Blue) = Low/Negative Correlation.**
  - **fmt='.2f': Tells the computer to show only 2 decimal places (e.g., 0.68).**

Slide 5: Code Breakdown (Cell 3 - The Pair Plot)

Code:

Python

```
# Step 4: Pair Plot

sns.pairplot(data, diag_kind="kde", plot_kws={'alpha':0.5})

plt.suptitle('Pair Plot of California Housing Features', y=1.02)

plt.show()
```

Explanation for Beginners:

- **sns.pairplot: This is a "Summary View." It creates a grid of scatter plots for *every* combination of columns.**
- **diag_kind="kde": On the diagonal (where a column meets itself), it draws a "wave" (Kernel Density Estimate) to show the distribution instead of a scatter plot.**
- **alpha=0.5: Makes the dots slightly see-through so you can see where they are most crowded.**

Slide 6: Key Vocabulary (The "Why")

Term	Easy Meaning
corr()	Finds the "link" between two columns using a score from -1 to 1.
Heatmap	A color-coded map. It lets you spot high numbers (strong links) instantly without reading every box.
Pair Plot	A "Galaxy View" of the data. It shows how every single feature interacts with every other feature.
Multicollinearity	(Advanced) When two features are too similar (e.g., total rooms and households).

Slide 7: Viva "Cheat Sheet" (Top Questions)

- **Q: Why do we use a Heatmap instead of just printing the numbers?**
  - **A: Visualizing with color helps us identify patterns and "hotspots" of high correlation much faster than looking at a table of numbers.**
- **Q: Which feature usually has the highest correlation with house value?**
  - **A: MedInc (Median Income). This proves that richer neighborhoods have higher house prices.**
- **Q: What is the benefit of a Pair Plot?**
  - **A: It allows us to see both the distribution of data (on the diagonal) and the relationships (scatter plots) in one single visualization.**

Is this Canva format helpful for your slides? If you are ready, let's move to Sl.NO 3 (PCA on the Iris Dataset).

This is the complete, line-by-line breakdown of your Experiment 3 code. I have organized it so you can copy the explanation directly into your Canva slides.

Sl.NO 3: Develop a program to implement Principal Component Analysis (PCA) for reducing the dimensionality of the Iris dataset from 4 features to 2.

Phase 1: The Imports (The Toolboxes)

Python

```
from sklearn.datasets import load_iris      # To get the flower data
from sklearn.decomposition import PCA       # The logic for "squashing" dimensions
from sklearn.preprocessing import StandardScaler # To make all measurements equal scale
import pandas as pd                         # To handle the data as a table
import matplotlib.pyplot as plt             # The basic drawing tool
```

Logic: We need StandardScaler because PCA is sensitive to size. If one column has large numbers (like 7.0) and another has small (like 0.2), the math will be biased.

Phase 2: Loading & Standardization

Python

```
iris = load_iris()          # Loads the 150 flower samples
features = iris.data        # This is the 4D data (Sepal/Petal size)
target = iris.target        # This is the ID (0, 1, or 2) for species

scaler = StandardScaler()   # Initializes the scaler
features_standardized = scaler.fit_transform(features)
```

What happened here? fit_transform calculates the average of every column and shifts everything so the Mean = 0 and Standard Deviation = 1. Now, every feature is "fairly" weighted.

Phase 3: The PCA Transformation

Python

```
pca = PCA(n_components=2)   # We tell PCA we want exactly 2 new columns
features_pca = pca.fit_transform(features_standardized)
```

The Math: The algorithm looks at the 4-dimensional cloud of data and finds the two angles (axes) that show the most spread. It "projects" the 4D points onto a flat 2D plane.

Phase 4: Organizing the Data

Python

```
pca_df = pd.DataFrame(data=features_pca, columns=["PC 1", "PC 2"])
pca_df["Target"] = target   # We add the target back so we know which dot is which flower
```

Logic: features_pca is just a list of numbers. By putting them into a pd.DataFrame, we create a clean table with named columns (PC 1 and PC 2).

Phase 5: The Visualization (Drawing the Graph)

Python

```
plt.figure(figsize=(8, 6))


for label, color in zip(iris.target_names, ["red", "green", "blue"]):
    plt.scatter(
        pca_df.loc[pca_df["Target"] == list(iris.target_names).index(label), "PC 1"],
        pca_df.loc[pca_df["Target"] == list(iris.target_names).index(label), "PC 2"],
        label=label,
        alpha=0.7
    )
```

Step-by-Step Logic:

1. **zip: This pairs each species name with a color: (Setosa → Red, Versicolor → Green, Virginica → Blue).**
2. **loc: This filters the table. It says: "Find all rows where the Target matches this specific species."**
3. **scatter: It draws the dots using the new PC 1 (x-axis) and PC 2 (y-axis) coordinates.**

Phase 6: Finalizing the Graph

Python

```
plt.title("PCA on Iris Dataset (4 to 2 features)")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend()                # Shows the box explaining Red, Green, Blue
plt.grid()                  # Adds the lines to the background
plt.show()                  # Actually opens the window to show the graph
```

Phase 7: The "Information Score" (Variance)

Python

explained_variance = pca.explained_variance_ratio_

print("Principal Component 1:", explained_variance[0])

print("Principal Component 2:", explained_variance[1])

print("Total Variance:", sum(explained_variance))

The Result:

- **PC 1: Captures the most information (usually ~72%).**
- **PC 2: Captures the next most (usually ~23%).**
- **Total Variance: Usually ~0.95 (95%). This means even though we deleted 2 dimensions, we kept 95% of the original information.**

Viva Cheat-Sheet (Canva Slide)

- **Why Standardize? PCA uses variance. Without scaling, a feature with large numbers would look more "important" to the math than a feature with small numbers.**
- **What are PC 1 and PC 2? They are not original features. They are "new" features created by combining the old ones to show the maximum spread of data.**
- **Which flower is easiest to find? Setosa (Red). In the graph, it is always an "island" far away from the other two.**
- **What is Total Variance? It is the accuracy score. It tells us how much of the original "story" of the data we were able to save.**

Perfect 👍 — this is a clean version of Find-S, so I'll explain it in Canva-ready format (clear + structured + easy to revise).

EXPERIMENT 4: FIND-S ALGORITHM – CODE EXPLANATION

AIM:For a given set of training data examples stored in a .CSV file, implement and demonstrate the Find-S algorithm to output a description of the set of all hypotheses consistent with the training examples.

1. Import Library

import pandas as pd

- **pandas is used to:**

- Read CSV file
- Handle tabular data

2. Load Dataset

data = pd.read_csv("training_data.csv")

print("Training Data Loaded:")

print(data)

print("-" * 30)

Explanation

- **Reads dataset from CSV file**
- **Displays dataset on screen**
- **"training_data.csv" must be in same folder**

👉 **Output: Table with features + target column**

3. Define Find-S Function

def find_s_algorithm(data):

- **Function to implement Find-S algorithm**
- **Takes dataset as input**

4. Extract Features & Target

attributes = data.iloc[:, :-1].values

target = data.iloc[:, -1].values

Explanation

- **attributes → all columns except last**
- **target → last column (Yes/No)**

👉 **Example:**

Experience	Skill	Age	Hired

- Features → Experience, Skill, Age
- Target → Hired

5. Initialize Hypothesis

```
hypothesis = None

for i in range(len(target)):

    if target[i] == "Yes":

        hypothesis = list(attributes[i])

        break
```

Explanation

- Start with first positive example
- Convert it to list → easier to modify

👉 Example:

['Senior', 'Java', 'Mid']

- break → stop after first "Yes"

6. Update Hypothesis

```
if hypothesis is not None:

    for i in range(len(target)):

        if target[i] == "Yes":

            for j in range(len(hypothesis)):

                if hypothesis[j] != attributes[i][j]:

                    hypothesis[j] = '?'
```

Explanation

- Compare hypothesis with other positive examples

👉 If mismatch:

- Replace value with '?'

Meaning of '?'

- **Represents any value allowed**
- **Used for generalization**

Example

Example	Experience	Skill	Age	Hired
1	Senior	Java	Mid	Yes
2	Junior	Java	Mid	Yes

👉 **Result:**

['?', 'Java', 'Mid']

7. Return Hypothesis

return hypothesis

- **Returns final generalized hypothesis**

8. Run Algorithm

final_hypothesis = find_s_algorithm(data)

- **Calls function**

9. Print Output

print("\nFinal Learned Most Specific Hypothesis:")

print(final_hypothesis)

- **Displays final result**

OUTPUT

Example:

Final Learned Most Specific Hypothesis:

['?', 'Java', '?', 'Young']

WORKING SUMMARY

1. Select first positive example
2. Compare with other positive examples
3. Generalize mismatched attributes
4. Ignore negative examples
5. Output final hypothesis

KEY POINTS (REVISION)

- Uses only positive examples
- '?' → represents generalization
- Produces single hypothesis
- Simple and fast algorithm

VIVA QUESTIONS

What is Find-S?
→ Finds most specific hypothesis from positive examples

Why ignore negative examples?
→ Algorithm only focuses on positive consistency

What does '?' mean?
→ Accepts any value

Limitation?
→ Cannot handle noisy or negative data

MEMORY LINE

👉 *Find-S = Start specific → Generalize using '?'*

If you want final help before exam:
🔥 **I can give:**

- 1-page revision for all 5 experiments
- OR most asked viva questions (very important)

I understand. I will go through every single line of the code for Experiment 4 without skipping anything, so you have a perfect record for your Canva slides and future externals.

Sl.NO 4: For a given set of training data examples stored in a .CSV file, implement and demonstrate the Find-S algorithm to output a description of the set of all hypotheses consistent with the training examples.

Purpose of the Dataset: training_data.csv

This dataset acts as the "Experience" for our machine.

- **It contains attributes like Experience, Qualification, and Skill.**
- **It has a target column "Hired" with "Yes" or "No".**
- **Find-S uses this to find the single most specific rule that explains why someone gets a "Yes".**

Phase 1: Preparation and Data Loading

Python

```
import pandas as pd
```

- **Line 1: We import the pandas library. This is the toolbox used to read CSV files and organize data into tables called DataFrames.**

Python

```
data = pd.read_csv("training_data.csv")
```

- **Line 2: We use the read_csv function to open the file "training_data.csv". The variable data now holds your entire spreadsheet.**

Python

```
print("Training Data Loaded:")

print(data)

print("-" * 30)
```

- **Line 3-5: These lines print the word "Training Data Loaded", show the actual table of data, and draw a line of dashes to make the output look organized.**

Phase 2: Defining the Find-S Logic (The Function)

Python

```
def find_s_algorithm(data):
```

- **Line 6: We define a function named find_s_algorithm. It takes your data table as an input.**

Python

```
    attributes = data.iloc[:, :-1].values
```

- **Line 7: iloc[:, :-1] tells the computer: "Take every row (colon) and every column EXCEPT the last one (-1)." These are your features (Experience, Qualification, etc.). .values converts them into a simple list.**

Python

```
target = data.iloc[:, -1].values
```

- **Line 8: iloc[:, -1] tells the computer: "Take every row but only the very last column." This is your Target (the "Yes" or "No" answers).**

Phase 3: Initializing the Hypothesis

Python

```
hypothesis = None
```

- **Line 9: We start by setting the hypothesis to None. This means the machine currently has "no rule" because it hasn't seen any data yet.**

Python

```
for i in range(len(target)):
    if target[i] == "Yes":
        hypothesis = list(attributes[i])
        break
```

- **Line 10-13: This loop looks through the "Hired" column row by row.**
- **If it finds the first "Yes", it takes that person's attributes and saves them as the starting hypothesis.**
- **break stops the loop immediately. We only need the first positive example to begin our rule.**

Phase 4: Updating the Hypothesis (The Learning Loop)

Python

```
if hypothesis is not None:
    for i in range(len(target)):
        if target[i] == "Yes":
```

- **Line 14-16: If we found at least one "Yes" candidate, we loop through the entire dataset again.**
- **We strictly check if target[i] == "Yes". Find-S completely ignores all "No" rows.**

Python

```
        for j in range(len(hypothesis)):
            if hypothesis[j] != attributes[i][j]:
                hypothesis[j] = '?'
```

- **Line 17-19: For every hired person, we compare their attributes with our current rule.**
- **If the attribute is different (e.g., the first person had "Python" and the second person had "Java"), we replace that spot in our rule with a ?.**
- **The ? means "Wildcard"—it tells the machine that for this specific category, any value is now acceptable.**

Phase 5: Finalizing and Printing

Python

```
    return hypothesis
```

- **Line 20: The function sends back the final, updated list of attributes (the hypothesis).**

Python

```
final_hypothesis = find_s_algorithm(data)
```

- **Line 21: We call the function and save the result into a variable called final_hypothesis.**

Python

```
print("\nFinal Learned Most Specific Hypothesis:")
print(final_hypothesis)
```

- **Line 22-23: We print the title and the final rule. For your data, it will look like ['Yes', '?', '?', '?'].**

Logic Summary for Your Viva

- **Most Specific: The algorithm starts very strict (only the first person) and slowly relaxes the rules only when it sees another "Yes" that doesn't fit.**
- **Positive Examples: The most important logic of Find-S is that it only learns from Positive (Yes) cases. It does not care about Negative (No) cases.**
- **The Question Mark (?): This indicates that the feature is not a requirement. In your result, having Experience=Yes is required, but the Skill or Age does not matter.**

Practical Questions for Internals

- **Q: What happens if there are no "Yes" entries in the CSV?**
- **A: The hypothesis will remain None, and the code will return no rule.**

- **Q: Why is Find-S called "Most Specific"?**
- **A: Because it finds the most restrictive rule that still covers all the successful examples. It doesn't allow any more flexibility than absolutely necessary.**
- **Q: Can this algorithm handle data errors?**
- **A: No. If a row is incorrectly labeled as "Yes," Find-S will add a ? and ruin the accuracy of the rule. This is a major limitation.**

I have covered every single line of the code you provided. Is this complete enough for your Canva presentation? We can move to Sl.NO 5 (k-NN) whenever you are ready.

Sl.NO 5: Develop a program to implement k-Nearest Neighbour algorithm to classify the randomly generated 100 values of x in the range of [0,1]. Perform the following based on dataset generated: a. Label the first 50 points; b. Classify the remaining points using k-NN for k=1, 2, 3, 4, 5, 20, 30.

1. Purpose of the Program: k-Nearest Neighbors (k-NN)

The **k-NN algorithm** is a "lazy learner." It doesn't learn a rule (like Find-S); instead, it just remembers the training data.

- **The Logic:** To classify a new point, it looks at the **k** closest neighbors. If most neighbors are "Blue," the new point becomes "Blue."
- **The Dataset:** We generate 100 random numbers between 0 and 1.
  - **Points 1–50:** Used for Training (we give them labels).
  - **Points 51–100:** Used for Testing (the machine must guess their labels).

2. Full Code Breakdown (Step-by-Step)

Phase 1: Imports and Data Generation

Python

```
import numpy as np

import matplotlib.pyplot as plt

from collections import Counter



# Generate 100 random numbers between 0 and 1

data = np.random.rand(100)



# Label first 50 points: if <= 0.5 then Class1, else Class2

# This creates our "Answer Key" for the machine to learn from
```

```
labels = ["Class1" if x <= 0.5 else "Class2" for x in data[:50]]
```

- **np.random.rand(100)**: Generates an array of 100 random floats.
- **Counter**: A special tool to count how many times "Class1" or "Class2" appears in the neighbors.

Phase 2: The Distance Logic

Python

```
def euclidean_distance(x1, x2):
    return abs(x1 - x2)
```

- **Logic:** In 1D math, the "distance" between two numbers is just the absolute difference.

Phase 3: The k-NN Classifier Function

Python

```
def knn_classifier(train_data, train_labels, test_point, k):
    # 1. Calculate distance from test_point to every training point
    distances = [(euclidean_distance(test_point, train_data[i]), train_labels[i])
                 for i in range(len(train_data))]

    # 2. Sort by distance (closest first)
    distances.sort(key=lambda x: x[0])

    # 3. Pick the 'k' closest neighbors
    k_nearest_neighbors = distances[:k]
    k_nearest_labels = [label for _, label in k_nearest_neighbors]

    # 4. Vote: return the label that appears most often
    return Counter(k_nearest_labels).most_common(1)[0][0]
```

- **distances.sort**: We sort the list so the points with the smallest distance (closest) are at the top.
- **most_common(1)**: This finds the "Winner" of the vote.

Phase 4: Running the Classification for Different k Values

Python

```
train_data = data[:50]
```

```
train_labels = labels

test_data = data[50:]


k_values = [1, 2, 3, 4, 5, 20, 30]

results = {}


for k in k_values:

    # Predict labels for all 50 test points

    classified_labels = [knn_classifier(train_data, train_labels, test_point, k)

                         for test_point in test_data]

    results[k] = classified_labels
```

Phase 5: Visualization

Python

```
# (Partial code for plotting)

plt.scatter(train_data, [0]*50, c=train_colors, marker="o", label="Train")

plt.scatter(test_data, [1]*50, c=test_colors, marker="x", label="Test")
```

- **Circle (o)**: Represents known training data at level 0.
- **Cross (x)**: Represents the machine's prediction at level 1.

3. Keywords Explained

Keyword	Definition	Logic in Lab
np.random.rand	**Uniform distribution generator.**	**Creates random values between 0 and 1.**
abs(x1 - x2)	**Absolute Difference.**	**The 1D version of Euclidean Distance.**
Counter	**Dictionary-based counter.**	**Counts the "Votes" of the neighbors.**
most_common(1)	**Frequency finder.**	**Finds the label with the highest count among neighbors.**
k	**Number of neighbors.**	**If $k=1$, the model is very specific. If $k=30$, the model is very smooth/general.**

4. Understanding the Result (The Logic)

1. **If $k=1$:** The machine is very sensitive. It looks at the single closest neighbor. If that neighbor is an outlier, the prediction might be wrong.
2. **If $k=30$:** The machine is very stable. It takes the average of 30 neighbors.
3. **The Decision Boundary:** You will notice that points near 0.5 are the hardest to classify. Because our rule was x <= 0.5, the machine tries to find that "boundary line" just by looking at the neighbors.

5. Viva Questions for Internals

- **Q: Is k-NN a Supervised or Unsupervised algorithm?**
  - **A:** Supervised, because it requires a "Training Set" with known labels to work.
- **Q: What happens if k is too small (e.g., $k=1$)?**
  - **A:** The model might suffer from **Overfitting** (it follows the "noise" in the training data too closely).
- **Q: What happens if k is too large (e.g., $k=50$)?**
  - **A:** The model might suffer from **Underfitting**. It will simply predict the majority class of the entire training set every time.
- **Q: Why do we prefer odd numbers for k (like 1, 3, 5)?**
  - **A:** To avoid "Ties" in voting. With an odd number, there is always a clear winner.

Is this breakdown ready for your Canva document? We can move to Sl.NO 6 (Locally Weighted Regression) next.

Program 6: Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs

(Book 1: Chapter – 4)

Here is the absolute line-by-line breakdown of the Python code, assuming zero prior Python or Machine Learning knowledge.

Part 1: The Setup (Imports)

```
Python

import numpy as np

import matplotlib.pyplot as plt
```

- **import numpy as np**: NumPy (Numerical Python) is a library that allows Python to do high-speed, advanced math on massive lists of numbers (called arrays/matrices). We shorten its name to np so we don't have to type numpy every time.
- **import matplotlib.pyplot as plt**: Matplotlib is Python's drawing tool. The pyplot module specifically helps us draw 2D graphs (like the scatter plot and lines at the end). We shorten it to plt.

Part 2: The Math Functions

```
Python

# Gaussian Kernel Function

def gaussian_kernel(x, xi, tau):

    return np.exp(-np.sum((x - xi) ** 2) / (2 * tau ** 2))
```

- **def gaussian_kernel(x, xi, tau):**: We are defining a custom function. It takes three inputs: x (the specific point we want to predict), xi (one of the training data points), and tau (the bandwidth/smoothing parameter).
- **np.sum((x - xi) 2)**: This calculates the squared distance between the point we want to predict and the training point. If they are far apart, this number is huge.
- **np.exp(...)**: This applies the exponential math function.
- **The Result**: Because of how this formula is built, if x and xi are very close together, the function returns a number very close to **1** (High Weight). If they are far apart, it returns a number very close to **0** (Low Weight).

```
Python

# Locally Weighted Regression Function

def locally_weighted_regression(x, X, y, tau):

    m = X.shape[0]
```

- **def locally_weighted_regression(...)**: The main brain of the program. It takes the point to predict (x), all the input data (X), all the target answers (y), and the smoothing parameter (tau).
- **m = X.shape[0]**: The .shape command tells us the dimensions of the dataset (Rows, Columns). Adding [0] tells Python to only grab the number of Rows. So, m is simply the total number of data points we have.

Python

```
    # Compute weights
    weights = np.array([gaussian_kernel(x, X[i], tau) for i in range(m)])
```

- **for i in range(m)**: This loops through every single data point in our dataset.
- **[gaussian_kernel(...)]**: For every point, it runs our math function to see how close it is to our target x. It generates a massive list of weights (mostly 0s, with a few 1s for the close neighbors).
- **np.array(...)**: Converts that regular Python list into a fast NumPy math list.

Python

```
    # Create diagonal weight matrix
    W = np.diag(weights)
```

- **np.diag(weights)**: To do matrix multiplication, our list of weights needs to be turned into a square grid (a matrix) where our weights run diagonally down the middle, and everything else is 0.

Python

```
    # Compute theta using weighted normal equation
    XTW = X.T @ W
    theta = np.linalg.pinv(XTW @ X) @ XTW @ y
    return x @ theta
```

- **X.T**: This transposes the matrix (flips it on its side).
- **@**: In Python, the @ symbol means "Matrix Multiplication".
- **np.linalg.pinv(...)**: This stands for "Linear Algebra Pseudo-Inverse". It calculates the inverse of the matrix. We use the "pseudo" version because it prevents the program from crashing if the math is imperfect.
- **theta**: This variable stores the calculated perfect slope and intercept for this specific local area.
- **return x @ theta**: It multiplies our input point by the calculated slope/intercept to give us the final predicted value!

Part 3: Preparing the Data

Python

```
# Generate dataset
np.random.seed(42)
```

```
X = np.linspace(0, 2 * np.pi, 100)

y = np.sin(X) + 0.1 * np.random.randn(100)
```

- **np.random.seed(42)**: Computers generate "fake" random numbers. Setting a seed is like saving a video game. It guarantees that every time the examiner runs your code, the "random" noise will look exactly the same.
- **np.linspace(0, 2 * np.pi, 100)**: Generates exactly 100 evenly spaced numbers between 0 and 2*Pi (6.28).
- **np.sin(X)**: Turns those numbers into a smooth wavy line (a Sine wave).
- **0.1 * np.random.randn(100)**: Generates 100 random numbers to act as "static" or "noise" and adds them to the wave so it looks like real-world, messy data.

Python

```
# Add bias term

X_bias = np.c_[np.ones(X.shape), X]
```

- **np.ones(X.shape)**: Creates a list filled purely with the number 1.
- **np.c_[...]**: This acts like glue. It takes the list of 1s and glues it as a new column right next to our X data.
- **Why?**: The math equation for a line is y = mx + c. The column of 1s allows the program to calculate c (the y-intercept). Without it, the line would be forced to cross at zero.

Part 4: Testing & Plotting

Python

```
# Test points

x_test = np.linspace(0, 2 * np.pi, 200)

x_test_bias = np.c_[np.ones(x_test.shape), x_test]

tau = 0.5
```

- **x_test**: We generate 200 new points to test the model on to see if it draws a smooth curve.
- **x_test_bias**: We glue the column of 1s onto our test data as well, just like we did with the training data.
- **tau = 0.5**: We set our bandwidth. 0.5 is a sweet spot for a smooth, wavy curve.

Python

```
# Predict values

y_pred = np.array([locally_weighted_regression(xi, X_bias, y, tau) for xi in x_test_bias])
```

- **What it does:** It takes our 200 test points, feeds them one-by-one (for xi in...) into the brain of our algorithm, and saves all 200 predictions into a new list called y_pred.

Python

```
# Plot results

plt.figure(figsize=(10, 6))
```

```
plt.scatter(X, y, color='red', label='Training Data', alpha=0.7)

plt.plot(x_test, y_pred, color='blue', label=f'LWR Fit (tau={tau})', linewidth=2)


plt.xlabel('X (Input Feature)')

plt.ylabel('y (Target Value)')

plt.title('Locally Weighted Regression')

plt.legend()

plt.grid(True)

plt.show()
```

- **plt.figure(figsize=(10, 6))**: Creates a blank canvas that is 10 inches wide and 6 inches tall.
- **plt.scatter(...)**: Takes our messy training data (X and y) and draws them as scattered Red dots. alpha=0.7 makes the dots slightly transparent.
- **plt.plot(...)**: Takes our 200 test predictions (x_test and y_pred) and draws a solid Blue line connecting them.
- **The rest**: Adds labels to the axes, puts a title on top, adds a legend box, turns on the background grid lines, and finally, plt.show() forces the pop-up window to appear on the screen!

3. The 5 Key Points to Memorize (For Exams & Vivas)

1. **Non-Parametric Nature:** LWR does not learn a fixed model; it must keep the entire training dataset in memory to make predictions, evaluating it newly every time.
2. **Local Fitting:** It fits a new linear regression line for *each* query point by only looking at data points close to that specific query.
3. **Gaussian Kernel:** Distance/importance is calculated using a bell-curve function. Closer points get a weight near 1, distant points get a weight near 0.
4. **The Bandwidth Parameter (tau):** Tau controls how wide the "neighborhood" is. Too small = overfitting (jagged line). Too large = underfitting (straight line).
5. **Weighted Normal Equation:** The model updates standard linear regression by injecting a weight matrix (W) into the formula to prioritize nearby points.

Program 7: Develop a program to demonstrate the working of Linear Regression and Polynomial Regression. Use Boston Housing Dataset for Linear Regression and Auto MPG Dataset for Polynomial Regression.

(Book 1: Chapter – 5)

Here is the absolute line-by-line breakdown of the Python code, assuming zero prior Python or Machine Learning knowledge.

Part 1: The Setup (Imports)

Python

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.preprocessing import PolynomialFeatures, StandardScaler

from sklearn.pipeline import make_pipeline

from sklearn.metrics import mean_squared_error, r2_score
```

- **import numpy as np**: Brings in NumPy for fast number crunching and matrix math.
- **import pandas as pd**: Pandas is the best tool for reading files (like Excel or CSVs). It organizes data into neat tables (called DataFrames).
- **import matplotlib.pyplot as plt**: Brings in our graphing and plotting tool.
- **from sklearn.model_selection import train_test_split**: Scikit-Learn (sklearn) is the ultimate Machine Learning library. This specific tool splits our data into a "study guide" (training data) and an "exam" (testing data).
- **from sklearn.linear_model import LinearRegression**: Imports the actual math brain for drawing straight lines.
- **from sklearn.preprocessing import PolynomialFeatures, StandardScaler**: PolynomialFeatures is the tool that gives our straight line the ability to bend (by squaring or cubing the inputs). StandardScaler makes sure all the numbers are scaled to a similar size so the math doesn't crash.
- **from sklearn.pipeline import make_pipeline**: A pipeline is like a factory assembly line. It allows us to snap multiple steps together easily.
- **from sklearn.metrics import ...**: Imports the grading tools (MSE and R^2) to see how well our AI learned.

Part 2: Linear Regression (The Straight Line)

Python

```
# --- PART 1: Linear Regression ---

def run_linear_regression():

    # Using the teacher-provided linear_dataset.csv

    data = pd.read_csv("linear_dataset.csv")
```

- **def run_linear_regression():**: Defines the start of our first program block.
- **data = pd.read_csv(...)**: Tells Pandas to open your teacher's CSV file and read the table of data.

Python

```
    X = data[["X"]].values

    y = data["Y"].values
```

- **X = data[["X"]].values**: Extracts the "X" column as our input (features). The double brackets make it a 2D matrix, which Scikit-Learn requires.
- **y = data["Y"].values**: Extracts the "Y" column as our target (the answer we want to predict).

Python

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- **train_test_split(...)**: This shuffles the data and hides 20% of it (test_size=0.2). The AI will train on 80% (X_train, y_train), and we will test it on the hidden 20% (X_test, y_test) to see if it actually learned or just memorized.
- **random_state=42**: Acts like a save state. It ensures the data is shuffled the exact same way every time you run it.

Python

```
model = LinearRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

- **model = LinearRegression()**: Creates a blank Machine Learning brain.
- **model.fit(X_train, y_train)**: The most important word in ML. "Fit" means "Train". The AI looks at the 80% data and calculates the perfect straight line ($y = mx + c$).
- **y_pred = model.predict(X_test)**: The exam! We feed the hidden 20% input data to the AI and ask it to predict the answers. We save its guesses in y_pred.

Python

```
plt.figure(figsize=(8, 5))

plt.scatter(X_test, y_test, color="blue", label="Actual Data")

plt.plot(X_test, y_pred, color="red", linewidth=2, label="Linear Fit")

# ... (labels and show commands) ...
```

- **What it does:** Uses Matplotlib to draw the actual hidden test data as Blue dots, and draws the AI's predicted straight line in Red.

Python

```
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))

print("R^2 Score:", r2_score(y_test, y_pred))
```

- **mean_squared_error**: Calculates how wrong the AI was on average. Lower is better.
- **r2_score**: The R^2 score is essentially an accuracy percentage. An R^2 of 1.0 is a perfect 100%.

Part 3: Polynomial Regression (The Curved Line)

Python

```
# --- PART 2: Polynomial Regression ---
```

```
def run_polynomial_regression():
    data = pd.read_csv("polynomial_dataset.csv")
    X = data[["X"]].values
    y = data["Y"].values
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- **What it does:** This is exactly the same as Part 2. It loads a new dataset (the curvy one) and splits it 80/20.

Python

```
    # Create a pipeline: Polynomial Features -> Scaling -> Linear Regression
    poly_model = make_pipeline(PolynomialFeatures(degree=3), StandardScaler(), LinearRegression())
    poly_model.fit(X_train, y_train)
```

- **make_pipeline(...)**: This creates our factory assembly line.
  - *Step 1:* PolynomialFeatures(degree=3) bends our data into a cubic curve (x^3).
  - *Step 2:* StandardScaler() shrinks the massive squared/cubed numbers down so the math is stable.
  - *Step 3:* Feeds that bent data into a standard LinearRegression model.
- **poly_model.fit(...)**: Trains this new curvy model on the 80% data.

Python

```
    # For a smooth curve in the plot, we create a range of values
    X_range = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
    y_range_pred = poly_model.predict(X_range)
```

- **np.linspace(...)**: To draw a smooth curve on the graph, we generate 100 tightly packed, perfectly ordered points from the minimum X value to the maximum X value. If we just connected the scattered test dots, the line would look like a messy zigzag instead of a smooth curve.
- **poly_model.predict(X_range)**: Asks the model to predict the answers for those 100 smooth points.

Python

```
    plt.scatter(X_test, y_test, color="blue", label="Actual Data")
    plt.plot(X_range, y_range_pred, color="red", linewidth=2, label="Polynomial Fit (Deg 3)")
    # ... (labels, show, and print metrics same as above) ...
```

- **What it does:** Draws the curvy test data as Blue dots, and draws the smooth Red curve using the 100 points we just generated. It then prints the MSE and R^2 scores.

Part 4: Executing the Code

```
```

```
Python

if __name__ == "__main__":

    run_linear_regression()

    run_polynomial_regression()
```

- **if __name__ == "__main__":**: This is a standard Python rule that says: "If the user runs this file directly, please automatically execute the two functions I just built above."

3. The 5 Key Points to Memorize (For Exams & Vivas)

1. **Purpose:** Linear Regression models straight-line relationships, while Polynomial Regression models non-linear (curved) relationships by raising inputs to a higher power (e.g., degree 2, degree 3).
2. **Train/Test Split:** The dataset is divided into training data (80%) to teach the model, and testing data (20%) to evaluate its performance on unseen data.
3. **Scikit-Learn Pipeline:** For polynomial regression, we use make_pipeline to cleanly chain together data transformations (PolynomialFeatures), scaling (StandardScaler), and the model (LinearRegression) in one single step.
4. **Data Scaling:** We use StandardScaler in polynomial regression because calculating x^2 or x^3 creates massive numbers that can destabilize the algorithm; scaling keeps the math stable.
5. **Evaluation Metrics:** The models are graded using Mean Squared Error (MSE), which measures the average squared difference between actual and predicted values, and the R^2 Score, which measures how well the model explains the variance (closer to 1.0 is better).

Program 8: Develop a program to demonstrate the working of the decision tree algorithm. Use Breast Cancer Data set for building the decision tree and apply this knowledge to classify a new sample.

(Book 2: Chapter – 3)

Here is the absolute line-by-line breakdown of the Python code, assuming zero prior Python or Machine Learning knowledge.

Part 1: The Setup (Imports)

```
Python

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.metrics import accuracy_score
```

- **import pandas as pd**: Used to load and handle the CSV data in a table format.
- **from sklearn.tree import DecisionTreeClassifier**: This is the "brain" of the program. It builds a tree-like model that makes decisions based on the data features.
- **from sklearn.tree import plot_tree**: A specific tool that draws the flowchart of the decision tree so we can see how the AI "thinks."
- **from sklearn.metrics import accuracy_score**: A grading tool to see what percentage of the test cases the AI got correct.

Part 2: Preprocessing the Data

Python

```
# 1. Load the dataset

df = pd.read_csv('Breast Cancer Dataset.csv')



# 2. Preprocessing

X = df.drop(['id', 'diagnosis'], axis=1)

y = df['diagnosis']
```

- **pd.read_csv(...)**: Loads the teacher-provided file into a DataFrame (table) called df.
- **df.drop(['id', 'diagnosis'], axis=1)**: We create our input features (X) by removing the "id" (useless for math) and "diagnosis" (which is the answer) columns.
- **y = df['diagnosis']**: We create our target variable (y) which contains the answers: Malignant (M) or Benign (B).

Python

```
# Convert categorical labels to numbers

y = y.map({'M': 1, 'B': 0})
```

- **y.map(...)**: Computers are bad at math with letters like "M" and "B." We convert "Malignant" to **1** and "Benign" to **0** so the algorithm can calculate probabilities.

Part 3: Training the "Tree"

Python

```
# 3. Split the data (80% training, 20% testing)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)



# 4. Train the Decision Tree Classifier

clf = DecisionTreeClassifier(max_depth=3, random_state=42)

clf.fit(X_train, y_train)
```

- **test_size=0.2**: We hide 20% of the data to test the AI later.
- **DecisionTreeClassifier(max_depth=3)**: We create the tree. max_depth=3 tells the tree it can only grow 3 levels deep. This prevents it from becoming too complex and unreadable.

- **clf.fit(...)**: The "Learning" phase. The tree looks at the 80% data and figures out which features (like cell size or texture) are the best "split points" to separate healthy cells from cancerous ones.

Part 4: Predictions & Classification

Python

```
# 5. Make predictions and check accuracy

y_pred = clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print(f"Model Accuracy: {accuracy * 100:.2f}%")
```

- **clf.predict(X_test)**: We give the hidden 20% of data to the AI and ask it to guess if the cells are 1 or 0.
- **accuracy * 100**: Prints out the final score (e.g., 95.00%).

Python

```
# 6. Classify a new sample

new_sample = X_test.iloc[[0]]

prediction = clf.predict(new_sample)



prediction_class = "Malignant" if prediction[0] == 1 else "Benign"

print(f"New Sample Classification Result: {prediction_class}")
```

- **X_test.iloc[[0]]**: To satisfy the manual requirement of "classifying a new sample," we grab the very first row from our test set.
- **prediction_class = ...**: We convert the numeric 1 or 0 back into a human-readable word for the output.

Part 5: Visualization

Python

```
# 7. Visualize the Decision Tree

plt.figure(figsize=(20, 10))

plot_tree(

    clf,

    feature_names=X.columns,

    class_names=['Benign', 'Malignant'],

    filled=True,
```

```
    rounded=True
)
plt.show()
```

- **plot_tree**: This draws the actual flowchart.
- **filled=True**: Colors the boxes (usually Blue for one class and Orange for the other) so you can visually see where the majority of samples lie.
- **feature_names**: Labels the "questions" at each node (e.g., "Is mean_radius > 14.5?").

3. The 5 Key Points to Memorize (For Exams & Vivas)

1. **Supervised Learning:** Decision Trees are a type of supervised learning algorithm used for classification by splitting data into subsets based on feature values.
2. **Entropy & Information Gain:** The tree decides where to split the data by calculating which feature reduces "uncertainty" (Entropy) the most—this is called Information Gain.
3. **The "Max Depth" Parameter:** We limit the depth of the tree to prevent "Overfitting," which happens when the model memorizes the training data too perfectly but fails on new, unseen data.
4. **Feature Importance:** Decision trees naturally rank features. The feature at the very top of the tree (the root) is considered the most important predictor in the dataset.
5. **Interpretability:** Unlike "Black Box" models (like Neural Networks), Decision Trees are highly interpretable because they can be visualized as a simple flowchart that a human can follow.

Program 9: Develop a program to implement the Naive Bayesian classifier considering Olivetti Face Data set for training. Compute the accuracy of the classifier, considering a few test data sets.

(Book 2: Chapter – 4)

Here is the absolute line-by-line breakdown of the Python code, assuming zero prior Python or Machine Learning knowledge.

Part 1: The Setup (Imports)

```
Python

import numpy as np

from sklearn.datasets import fetch_olivetti_faces

from sklearn.model_selection import train_test_split, cross_val_score

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score, classification_report

import matplotlib.pyplot as plt
```

- **fetch_olivetti_faces**: This is a famous dataset of 400 faces (10 different images of 40 different people). We use it to teach the AI to recognize who is who.

- **GaussianNB**: This is the "Naive Bayes" algorithm brain. The "Gaussian" part means it assumes the data (pixel values) follows a normal distribution (bell curve).
- **cross_val_score**: A tool that runs the training multiple times on different parts of the data to make sure the accuracy wasn't just a "lucky" guess.
- **classification_report**: Gives a detailed breakdown of which faces the AI recognized easily and which ones it struggled with.

Part 2: Loading & Splitting Data

Python

```
# 1. Load the dataset

data = fetch_olivetti_faces(shuffle=True, random_state=42)

X = data.data  # The actual image pixels (input)

y = data.target  # The ID number of the person (answer)



# 2. Split (70% training, 30% testing)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

- **data.data**: These are the features. Each face is 64x64 pixels, flattened into a long list of 4,096 numbers representing brightness.
- **data.target**: These are the labels. Each person is assigned a number from 0 to 39.
- **test_size=0.3**: This tells the program to use 70% of the photos for teaching the AI and keep 30% hidden for the final exam.

Part 3: Training the AI

Python

```
# 3. Initialize and train the classifier

gnb = GaussianNB()

gnb.fit(X_train, y_train)
```

- **GaussianNB()**: Creates the empty Naive Bayes brain.
- **gnb.fit(...)**: The training phase.
  - **How it works:** For every person (ID), it calculates the average "brightness" of every single pixel.
  - **The "Naive" part:** It assumes that the brightness of one pixel doesn't depend on the pixel next to it.

Part 4: Evaluation & Accuracy

Python

```
# 4. Make predictions and check accuracy

y_pred = gnb.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
```

```
print(f'Accuracy: {accuracy * 100:.2f}%')


# 5. Classification Report (Detailed stats)
print(classification_report(y_test, y_pred, zero_division=1))


# 6. Cross-validation
cross_val_accuracy = cross_val_score(gnb, X, y, cv=5)
print(f'Cross-validation (Average): {cross_val_accuracy.mean() * 100:.2f}%')
```

- **gnb.predict(X_test)**: The AI looks at the 30% hidden photos and tries to guess which ID number they belong to.
- **cross_val_score(..., cv=5)**: It splits the data into 5 parts. It trains on 4 parts and tests on 1, then repeats this 5 times. It calculates the average of all 5 scores to give a more honest accuracy percentage.

Part 5: Visualization

Python

```
# 7. Visualize the results
fig, axes = plt.subplots(3, 5, figsize=(12, 8))
for i, ax in enumerate(axes.ravel()):
    # Show the image (Reshape 4096 back to 64x64)
    ax.imshow(X_test[i].reshape(64, 64), cmap=plt.cm.gray)

    # Set title with True label and Predicted label
    title_color = "green" if y_test[i] == y_pred[i] else "red"
    ax.set_title(f"True: {y_test[i]}\nPred: {y_pred[i]}", color=title_color, fontsize=10)
    ax.axis('off')

plt.tight_layout()
plt.suptitle("Face Recognition Results (Naive Bayes)", fontsize=16, y=1.05)
plt.show()
```

- **plt.subplots(3, 5)**: Creates a grid of 15 small windows (3 rows, 5 columns) to display faces.
- **ax.imshow(..., cmap=plt.cm.gray)**: Takes the 4,096 numbers, reshapes them back into a 64x64 square, and draws them as a grayscale face.

- **title_color**: If the AI guessed correctly, the text is **Green**. If it failed, the text is **Red**.
- **plt.show()**: Forces the window with all the faces and titles to appear on the screen.

3. The 5 Key Points to Memorize (For Exams & Vivas)

1. **Bayes' Theorem:** The algorithm is based on calculating the probability of a label (person's ID) given the input features (pixel values).
2. **The "Naive" Assumption:** It assumes that all features (pixels) are independent of each other. In face recognition, it assumes the brightness of your eye doesn't relate to the brightness of your nose.
3. **Gaussian Distribution:** It assumes that the continuous data (pixel intensity) follows a Gaussian (normal) distribution, allowing it to calculate probabilities using the mean and variance.
4. **Feature Vector:** Each 64x64 image is flattened into a 1D array of 4,096 features, where each feature represents a pixel's brightness between 0 and 1.
5. **Suitability:** Naive Bayes is extremely fast and requires less training data than complex models, but its accuracy can be lower in face recognition because pixel values are actually highly dependent on their neighbors.

Program 10: Develop a program to implement k-means clustering using Wisconsin Breast Cancer data set and visualize the clustering result.

Here is the absolute line-by-line breakdown of the Python code you provided, assuming zero prior Python or Machine Learning knowledge.

Part 1: The Setup (Imports)

```
Python
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.datasets import load_breast_cancer
```

```
from sklearn.cluster import KMeans
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.decomposition import PCA
```

```
from sklearn.metrics import confusion_matrix, classification_report
```

- ***from sklearn.datasets import load_breast_cancer***: *This loads a built-in dataset containing 569 samples of cancer cells with 30 different physical measurements (like radius, texture, and area).*
- ***from sklearn.cluster import KMeans***: *This is the "Unsupervised" algorithm. It doesn't look at the answers; it just groups points based on how close they are to each other.*

- ***from sklearn.preprocessing import StandardScaler**: **CRITICAL:** K-Means calculates distance. If one feature has huge numbers (e.g., Area = 1000) and another has tiny numbers (e.g., Smoothness = 0.1), the huge numbers will overpower the math. Scaling makes all features equal in importance.*
- ***from sklearn.decomposition import PCA**: Principal Component Analysis. We cannot draw a 30-dimensional graph. PCA squashes those 30 columns into 2 "Principal Components" (PC1 and PC2) so we can see the clusters on a 2D plot.*

Part 2: Loading and Scaling Data

Python

```
# Load dataset

data = load_breast_cancer()

X = data.data  # Features (the 30 measurements)

y = data.target  # True labels (0 for Malignant, 1 for Benign)



# Standardize data

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

- ***X = data.data**: Stores the raw measurements.*
- ***y = data.target**: We store the real answers here. We won't give them to the AI while it's learning, but we use them at the very end to see if the AI's clusters match reality.*
- ***scaler.fit_transform(X)**: Standardizes the data so the mean is 0 and the standard deviation is 1.*

Part 3: The K-Means "Clustering"

Python

```
# Apply K-Means

kmeans = KMeans(n_clusters=2, random_state=42)

y_kmeans = kmeans.fit_predict(X_scaled)
```

- ***n_clusters=2**: We tell the AI to find exactly two groups (clusters).*
- ***fit_predict(...)**: This does two things:*
  1. ***Fit**: The AI places two "Centroids" (starting points) and moves them until they are in the geometric center of two distinct groups.*
  2. ***Predict**: It gives every data point a label (0 or 1) based on which Centroid it is closest to.*

Part 4: Squashing Data for Visualization (PCA)

Python

```
# PCA for visualization
```

```
pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)


# Create DataFrame

df = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])

df['Cluster'] = y_kmeans

df['True Label'] = y
```

- ***PCA(n_components=2)***: *This "squashes" the 30-dimensional data into 2 dimensions.*
- ***pd.DataFrame***: *We build a table where PC1 and PC2 are the (x, y) coordinates for our graph, while Cluster is the AI's group guess and True Label is the actual diagnosis.*

Part 5: Visualization with Centroids

```
Python

# Plot with Centroids

plt.figure(figsize=(8, 6))

sns.scatterplot(data=df, x='PC1', y='PC2', hue='Cluster', palette='Set1', alpha=0.7)


# Transform centroids to PCA space

centers = pca.transform(kmeans.cluster_centers_)

plt.scatter(centers[:, 0], centers[:, 1], s=200, c='red', marker='X', label='Centroids')


plt.title('K-Means Clustering with Centroids')

plt.legend(title="Cluster")

plt.show()
```

- ***hue='Cluster'***: *Colors the dots based on the groups found by the AI.*
- ***kmeans.cluster_centers_***: *This gives the coordinates of the "Heads" (Centroids) of the clusters.*
- ***pca.transform(...)***: *Because the graph is in 2D (PCA space), we must also convert the 30-dimensional centroids into 2D so they can be drawn.*
- ***marker='X', c='red'***: *Draws a large **Red X** at the center of each group.*
- ***plt.legend()***: *Adds the box explaining what the colors and the "X" represent.*

3. The 5 Key Points to Memorize (For Exams & Vivas)

1. ***Unsupervised Learning:*** *K-Means is an unsupervised algorithm, meaning it discovers groups (clusters) in data without needing to know the "correct answers" during the training process.*

2. ***Centroids & Iteration:*** *The algorithm works by placing K center points (Centroids) and iteratively moving them until they are in the center of their respective clusters.*
3. ***Distance Dependency:*** *K-Means relies on the Euclidean distance between points. This makes* ***Standardization (Scaling)*** *mandatory so that features with large ranges don't dominate the clustering.*
4. ***PCA for Visualization:*** *Since humans cannot visualize 30 dimensions, we use* ***Principal Component Analysis (PCA)*** *to reduce the data to two dimensions (PC1 and PC2) while preserving the data's variance.*
5. ***Inertia:*** *The goal of K-Means is to minimize the sum of squared distances between data points and their assigned centroids, a value often referred to as "Inertia" or "Within-Cluster Sum of Squares (WCSS)."*